

3교시: Logs와 HTTP 정상 확인

수업 목표

- docker ps의 Up과 서비스 정상 응답을 구분한다.
- docker logs에서 startup/access/error 신호를 확인한다.
- HTTP 응답으로 사용자가 접근 가능한 상태를 확인한다.

개념 설명

container가 Up이면 process가 살아 있다는 뜻이다. 하지만 사용자가 접속 가능한지, 올바른 port로 열렸는지, app이 정상 응답하는지는 별도 확인이 필요하다. 그래서 logs와 HTTP 확인을 같이 본다.

nginx는 좋은 예시다. process가 떠 있고 port publish가 맞으면 curl -I에서 HTTP/1.1 200 OK가 나온다. 반대로 port를 잘못 보면 container는 Up인데 host에서는 접속이 실패한다.

logs를 볼 때도 환경 설정과 secret 기준을 같이 본다. 예를 들어 APP_ENV=staging으로 실행한 서비스가 startup log에 mode=staging 정도를 남기는 것은 도움이 된다. 하지만 DB_PASSWORD=... 같은 실제 secret을 log에 찍으면 실패다. 로그는 장애 분석을 위한 증거이면서 동시에 유출 경로가 될 수 있다.

실습 명령

```
cd /mnt/d/paperclip
docker rm -f paperclip-day4-nginx || true
docker run -d --name paperclip-day4-nginx -p 18084:80 nginx:1.27-alpine
docker ps --filter name=paperclip-day4-nginx
docker logs paperclip-day4-nginx --tail 30
curl -I http://localhost:18084
docker logs paperclip-day4-nginx --tail 30
```

Expected:

```
STATUS Up
0.0.0.0:18084->80/tcp
HTTP/1.1 200 OK
```

확인 지점

증거	의미
STATUS Up	container process가 실행 중
0.0.0.0:18084->80/tcp	host 18084가 container 80으로 연결
HTTP/1.1 200 OK	host에서 서비스 접근 성공
access log	HTTP 요청이 container까지 도달

log에 남겨도 되는 것과 안 되는 것

로그 예시	판단
APP_ENV=staging	환경 이름 정도는 가능
HTTP/1.1 200 OK	정상 확인 증거
GET /	접근 확인 증거
DB_PASSWORD=my-real-password	실패, secret 노출
AWS_SECRET_ACCESS_KEY=...	실패, credential 노출

오해 교정

docker logs가 비어 있다고 항상 장애는 아니다. 요청을 아직 보내지 않았거나, image가 startup log를 적게 남길 수 있다. 이때는 curl을 먼저 보내고 logs를 다시 본다.

환경별 파일을 쓰는 서비스라면 logs에는 어느 환경으로 뺐는지를 확인할 힌트가 남을 수 있다. 다만 password 자체를 확인하려고 logs에 찍는 방식은 금지한다. 값 확인은 masking된 script, inspect, application health endpoint 등으로 제한한다.

다음 연결

다음 교시는 inspect와 exec로 Docker metadata와 container 내부 상태를 나눠 본다.